

Proyecto Práctico: App con Navegación

Objetivo

Construir una aplicación con navegación completa que demuestre el uso de Stack Navigator, Tab Navigator y los principios de Material Design de forma práctica.

Conceptos Fundamentales de Navegación

Para que las pantallas se comuniquen entre sí, React Navigation nos provee dos objetos principales a través de las "props" de cada pantalla:

1. navigation (El "Control Remoto")

Es un objeto que contiene funciones para disparar acciones de navegación. Las más comunes son:

- `navigation.navigate('NombreRuta', { params })`: Nos lleva a una pantalla específica. Si ya estamos ahí, no hace nada.
- `navigation.push('NombreRuta')`: Añade una nueva pantalla a la pila, permitiendo ir a la misma pantalla varias veces (ej: de un perfil a otro perfil).
- `navigation.goBack()`: Cierra la pantalla actual y vuelve a la anterior.
- `navigation.replace('NombreRuta')`: Reemplaza la pantalla actual por una nueva (útil para el Login).

2. route (La "Información de Viaje")

Es un objeto que contiene información sobre la ruta actual. Lo más importante aquí es `route.params`.

3. route.params (El "Equipaje")

Es un objeto que contiene los datos que fueron pasados a la pantalla al navegar.

- En la pantalla de origen (envío): `navigation.navigate('Details', { item: { id: 1, name: 'Producto' } })`
- En la pantalla de destino (recepción): `const { item } = route.params;`

🤔 ¿De dónde salen navigation y route? (La "Magia")

Esta es una duda muy común al empezar. ¿Quién le "pasa" estas variables a mis componentes si yo no lo hago manualmente?

El Navegador es el "Padre"

Cuando registras una pantalla en un Navegador (Stack o Tab) así:

```
<Stack.Screen name="Home" component={HomeScreen} />
```

El componente principal de React Navigation (el Stack.Navigator) "envuelve" a tu componente HomeScreen. Al hacerlo, inyecta automáticamente estas props (navigation y route) dentro de él.

¿Cómo las atrapamos? (Destructuring)

Como son objetos que vienen en las "props" del componente, las recibimos en los parámetros de la función usando llaves {}:

```
export default function HomeScreen({ navigation, route }) {
```

```
  // Ahora podemos usarlas aquí adentro
```

```
}
```

¿Qué pasa si mi componente NO es una pantalla?

Si creas un componente pequeño (como un Botón personalizado) y quieres que navegue, no recibirá estas props automáticamente porque no está registrado directamente en el Navigator. Para estos casos, React Navigation nos da "Hooks":

- `useNavigation()`: Para obtener el objeto navigation.
 - `useRoute()`: Para obtener el objeto route.
-

Componentes/Conceptos a Aplicar

- **Stack Navigator:** Navegación entre pantallas en flujo lineal (una encima de otra).
- **Tab Navigator:** Navegación principal por pestañas (generalmente abajo).
- **React Native Paper:** Librería de componentes siguiendo Material Design 3.
- **Navegación anidada:** Meter un Stack dentro de un Tab para flujos complejos.



Pasos Guía para el Desarrollo

1. **Estructura Base:** Organiza tus carpetas para que el código sea fácil de encontrar.

- **app/screens:** Tus pantallas (Home, Details, etc).
- **app/navigation:** Donde configuras tus Navigators.
- **app/components:** Elementos reutilizables como el ScreenWrapper.

2. **Configurar el Layout Principal (_layout.tsx):**

- Envuelve toda la app en **PaperProvider** (para estilos Material) y **SafeAreaProvider** (para evitar notch/barras del sistema).
- Coloca tu Navegador raíz (ej: **TabNavigator**) aquí.

3. **Definir el Stack de Inicio (HomeStack.tsx):**

- Crea un **createNativeStackNavigator**.
- Registra la pantalla de lista (Home) y la de detalle (Details).
- **Tip:** Desactiva el **headerShown** si vas a usar un componente personalizado para el título.

4. **Definir el Tab Navigator (TabNavigator.tsx):**

- Crea el menú inferior.

- En lugar de poner la pantalla Home directamente, pon el HomeStack. Esto permite que cuando estés en la pestaña de inicio, puedas "entrar" a los detalles y seguir viendo el menú de abajo si así lo deseas.

5. Implementar la Navegación con Parámetros:

- En HomeScreen, usa un FlatList para mostrar datos.
- Al tocar un item, usa `navigation.navigate('Details', { item })`.
- En DetailsScreen, recupera los datos con `const { item } = route.params`.

Mejores Prácticas y Consejos

Navegación

- **Navegación Intuitiva:** Asegúrate de que el usuario siempre pueda volver atrás (usar `navigation.goBack()` o el botón nativo).
- **Tipado:** Si usas TypeScript, define los tipos de tus rutas para evitar errores al pasar parámetros.
- **Nivel de Anidación:** No metas un Stack dentro de otro Stack dentro de otro Tab. Mantén la estructura simple (Máximo 2 niveles).

Material Design (UI/UX)

- **Espaciado:** Usa múltiplos de 8px (8, 16, 24, 32) para márgenes y paddings. Da una sensación de orden.
- **Feedback:** Si el usuario toca un botón o card, debe haber un efecto visual (ripple/sombra).
- **Consistencia:** Usa `sharedStyles.ts` para que todas las pantallas se vean parecidas.

Recursos para profundizar

- Docs Oficiales de React Navigation: [Navigate Method](#)

- **Guía de Parámetros:** [Passing Parameters to Routes](#)
- **React Native Paper Components:** [Button](#)
- **Material Design 3 Guidelines:** [M3 Layout](#)